

# Hermes Agent Skill 自进化机制

## 源码证据版学习文档

聚焦：类别归属 · 源码提示词标准 · 新建/修改判定 · Background Review 流程

适用场景：研究 Hermes 的 Skill 表示、自学习、自进化与程序性记忆机制

核心结论：

Hermes 的自进化不是训练模型参数，而是把可复用任务经验写回 Skill 文件系统。

新经验优先 patch 本轮已用 Skill；其次 patch 已有 umbrella Skill；再写 references/templates/scripts；最后才 create 新的 class-level Skill。

类别归属不是硬编码分类器决定，而是由 Review/Learn Prompt 指导 LLM 选择 sensible category，源码只做名称、Frontmatter、路径、安全与写入校验。

0. 阅读定位与结论总览

本文专门回答三个容易混淆的问题：

- 新沉淀的 Skill 如何判断属于已有类别，还是需要新建类别？
- Hermes 源码提示词对 Skill 写作、学习信号、禁止沉淀内容有哪些标准？
- 具体评判机制如何决定“新建 Skill”还是“修改原有 Skill”？

**最重要的结论是：Hermes 没有把这些判断做成硬编码分类器或训练好的 Skill Router；大部分语义判断由 Background Review Agent 根据源码中的 Prompt 策略完成，skill\_manager\_tool 负责执行写入、校验和安全边界。**

| 问题          | 源码层机制                                                                 | 不是                    |
|-------------|-----------------------------------------------------------------------|-----------------------|
| 类别归属        | LLM 根据 prompt 选择 sensible category；skill_manage 只校验 category 是合法单级目录。 | 不是硬编码类别分类器。           |
| 是否修改旧 Skill | 优先本轮已加载 Skill；其次已有 umbrella；再写支持文件。                                   | 不是遍历全部 Skill 做全库判定。   |
| 是否新建 Skill  | 没有已有 class-level Skill 覆盖时，才 create 新 umbrella Skill。                 | 不是每个新经验都创建一个 Skill。   |
| 自进化对象       | SKILL.md、references、templates、scripts、assets 等文件。                     | 不是模型权重或 embedding 参数。 |

1. Hermes 自进化的定义

Hermes 的 Skill 自进化可以定义为：Agent 在任务执行、用户纠正和工具轨迹中识别可复用程序性经验，并通过 skill\_manage 将这些经验写回 Skill 文件系统，使后续会话可通过 available\_skills、skill\_view 或 Bundle 加载更新后的能力。



因此，“Agent grows with you”的关键不是模型参数变化，而是外部程序性知识库变化。

2. 新沉淀 Skill 的类别归属机制

2.1 源码结论：没有硬编码类别分类器

Hermes 没有专门的 category classifier。类别归属主要由 LLM 在 /learn 或 Background Review 的提示词引导下选择，skill\_manager\_tool 的职责是接收 category、校验其格式，并把新 Skill 写入对应目录。

| 源码证据                                                                                    | 含义                          |
|-----------------------------------------------------------------------------------------|-----------------------------|
| learn_prompt.py 要求 “Author ONE SKILL.md ... Pick a sensible category.”                  | 说明类别由 Agent 作者语义选择，而非算法分类器。 |
| skill_manager_tool.py 的 category schema：Optional category/domain；Only used with create。 | 说明 category 是写入组织字段。        |

|                                                                 |                     |
|-----------------------------------------------------------------|---------------------|
| _validate_category 只检查单级目录名是否合法。                                | 说明源码只做格式校验，不判断语义归属。 |
| _resolve_skill_dir(name, category) 将目录解析为 skills/category/name。 | 说明类别最终体现为文件系统目录。    |

- 新 Skill 类别选择的真实链路：
- 1. Review/Learn Agent 读任务经验与现有 Skill。
  - 2. 判断是否已有 Skill/umbrella 能承载。
  - 3. 若必须创建新 Skill，由 LLM 选择 class-level name 与 sensible category。
  - 4. skill\_manage(create, name, category, content) 只做格式、Frontmatter、路径与安全校验。
  - 5. 落盘到 ~/.hermes/skills/<category>/<skill-name>/SKILL.md。

研究表述建议：Hermes 的类别归属是“prompt-guided semantic placement + filesystem validation”，不是“learned category classifier”。

3. 已有类别与已有 Skill 的归属判定

3.1 优先级不是“新建优先”，而是“复用/扩展优先”

Background Review 的核心策略是 class-first：目标库形态应是少量 class-level umbrella Skill，而不是大量 one-session-one-skill 条目。因此新经验首先尝试归入本轮已用 Skill 或已有 umbrella。

| 优先级 | 动作                                | 含义                                                                                  |
|-----|-----------------------------------|-------------------------------------------------------------------------------------|
| 1   | UPDATE A CURRENTLY-LOADED SKILL   | 如果本轮 /skill-name 或 skill_view 读过的 Skill 覆盖该经验，优先 patch 它。                           |
| 2   | UPDATE AN EXISTING UMBRELLA       | 如果本轮 Skill 不合适，但已有 class-level Skill 覆盖该任务类别，使用 skills_list + skill_view 找到并 patch。 |
| 3   | ADD A SUPPORT FILE                | 如果细节太长或太具体，写入 references/templates/scripts，并在 SKILL.md 加一行指针。                       |
| 4   | CREATE A NEW CLASS-LEVEL UMBRELLA | 只有没有任何已有 Skill 覆盖该 class 时，才创建新 Skill。                                              |

- 判定路径：
- 新经验 → 是否属于本轮已加载 Skill？
- └ 是：patch 当前 Skill
  - └ 否：是否已有 umbrella Skill 覆盖？
    - └ 是：patch umbrella 或 write\_file 到 references/templates/scripts
    - └ 否：create 新 class-level umbrella Skill

4. Hermes 源码提示词中的 Skill 作者标准

4.1 Frontmatter 标准

| 字段          | 标准                                                  |
|-------------|-----------------------------------------------------|
| name        | 小写、连字符风格，长度不超过 64 字符，不能有空格。                         |
| description | 一句话，不超过 60 字符，以句号结尾；描述能力而不是实现；不能使用营销词，不能重复 Skill 名。 |
| version     | 新建 Skill 默认使用 0.1.0。                                |
| author      | 必须写字面值 Hermes，不能从系统用户名、git config 或环境变量推断。          |
| platforms   | 只有 Skill 真的依赖 OS 特性时才声明；可移植 Skill 应省略。              |

|                      |                   |
|----------------------|-------------------|
| metadata.hermes.tags | 使用少量相关标签，用于组织和阅读。 |
|----------------------|-------------------|

**description 的 60 字符限制不是美观要求，而是路由机制要求：system prompt 中的 Skill index 会截断短描述，超过部分不会参与常驻路由。**

4.2 正文结构标准

| 顺序 | 章节              | 作用                      |
|----|-----------------|-------------------------|
| 1  | # 标题 + 简介       | 说明能力边界、不会做什么、关键依赖。      |
| 2  | When to Use     | 列出具体触发条件和用户表达。          |
| 3  | Prerequisites   | 环境变量、安装步骤、凭据要求。         |
| 4  | How to Run      | 用 Hermes 工具视角写标准调用方式。   |
| 5  | Quick Reference | 命令、端点、路径的速查表。           |
| 6  | Procedure       | 编号步骤与可复制命令。             |
| 7  | Pitfalls        | 已知限制、速率限制、看似坏了但不是坏了的情况。 |
| 8  | Verification    | 一个能证明 Skill 生效的检查。      |

4.3 Hermes 工具表达标准

源码提示词要求 Skill 文档从 Hermes 工具视角写，而不是普通 shell 教程。应写 read\_file、search\_files、patch、web\_extract、terminal 等工具名称；不要用 cat/head/tail、grep/rg/find/ls、sed/awk 来替代 Hermes 已经封装的工具语义。

推荐：使用 `read\_file` 读取配置，使用 `search\_files` 查找引用，使用 `patch` 修改目标片段。  
不推荐：用 cat/head/tail、grep/rg/find、sed/awk 描述同类能力。

5. 学习信号：什么时候值得沉淀 Skill

Background Review 的提示词把几类现象定义为“值得行动”的学习信号。只要出现一个，就应该考虑更新 Skill 或 Memory，但仍需判断是否是可复用经验。

| 学习信号                                       | 应如何处理                              |
|--------------------------------------------|------------------------------------|
| 用户纠正风格、格式、可读性、详细程度                         | 更新管理该类任务的 Skill；不能只存 Memory。       |
| 用户纠正工作流、方法或步骤顺序                            | 作为 Procedure 或 Pitfall 写入相关 Skill。 |
| 出现非平凡技术、修复、绕过方案、debug pattern、tool pattern | 沉淀为可复用流程，优先补充已有 Skill。             |
| 本轮加载的 Skill 错了、缺步骤或过时                      | 立即 patch 本轮使用过的 Skill。             |

6. 禁止沉淀的内容

| 禁止类型                 | 原因                                                                                   |
|----------------------|--------------------------------------------------------------------------------------|
| 环境依赖的临时故障            | 例如 missing binaries、fresh install errors、command not found、credentials 未配置；这些不是长期规则。 |
| 对工具或功能的负面结论          | 例如 “browser tool 不工作” “X tool broken”；会变成长期自我限制。                                     |
| 会话内 transient errors | 如果 retry 后成功，应保存 retry pattern，而不是保存原始失败。                                            |
| 一次性任务叙事              | 例如 “分析今天市场” “分析这个 PR”；不是可复用类别。                                                       |

7. 新建 Skill 还是修改原有 Skill：具体评判机制

7.1 skill\_manager\_tool 的动作标准

| 动作          | 源码标准 / 使用时机                                                    |
|-------------|----------------------------------------------------------------|
| create      | 复杂任务成功、5+ calls、克服错误、用户纠正方法生效、发现非平凡 workflow、用户要求记住 procedure。 |
| patch       | old_string/new_string 局部替换；用于修复步骤、补 pitfall、补触发条件，是 fixes 的首选。 |
| edit        | 完整重写 SKILL.md；只用于 major overhaul。                              |
| write_file  | 添加/覆盖支持文件，限 references/templates/scripts/assets。               |
| remove_file | 删除支持文件。                                                        |
| delete      | 删除或归档；合并时需 absorbed_into 指向已存在 umbrella。                       |

特别注意：如果本轮使用了某个 Skill，并且遇到了它未覆盖的问题，源码工具描述要求 “patch it immediately”。这说明 Hermes 的默认方向是改旧 Skill，而不是新建一个碎片 Skill。

7.2 判定算法式总结

```
输入：本轮会话、工具轨迹、用户纠正、已加载 Skill、skills_list/skill_view 结果

if 没有学习信号：
    Nothing to save
elif 信号是事实性偏好或用户信息：
    写 Memory
elif 本轮已加载 Skill 覆盖该经验：
    skill_manage(patch/edit/write_file, target=loaded_skill)
elif 已有 umbrella Skill 覆盖该类别：
    skill_manage(patch/write_file, target=umbrella_skill)
elif 经验是细节、模板或脚本：
    write_file 到 references/templates/scripts，并在 SKILL.md 添加指针
else:
    create 新 class-level umbrella Skill，选择 sensible category
```

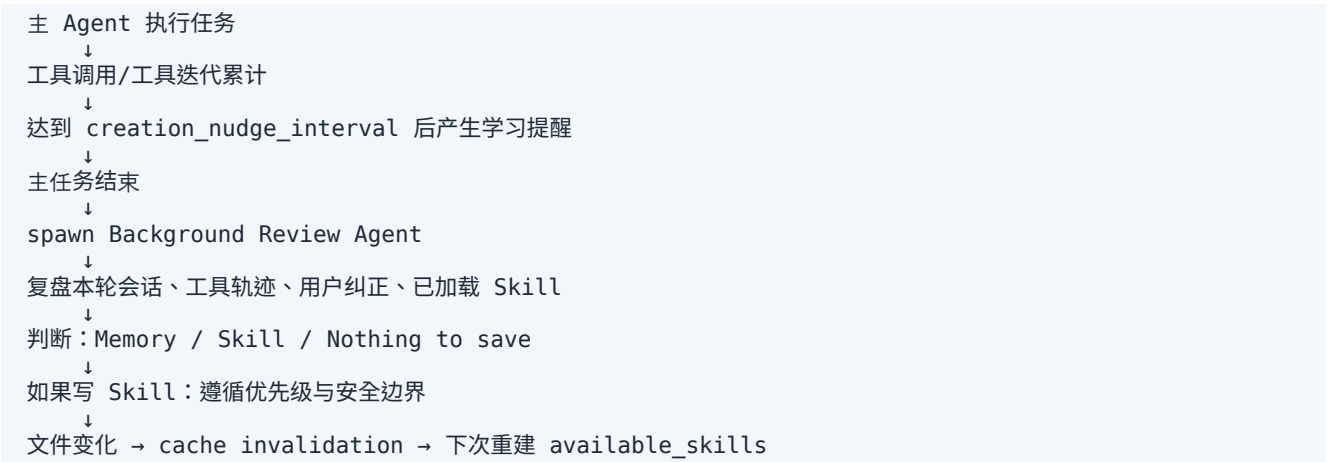
8. 支持文件的使用策略

支持文件是防止 SKILL.md 过度膨胀的关键。Hermes Prompt 明确区分三类主要支持文件：references、templates、scripts。

| 目录          | 适合内容                                                           | 示例                         |
|-------------|----------------------------------------------------------------|----------------------------|
| references/ | 会话细节、错误 transcript、复现步骤、provider quirks、研究摘录、API 文档要点、领域笔记。    | references/uv-workflow.md  |
| templates/  | 可复制后修改的 starter files、boilerplate configs、known-good examples。 | templates/debug-report.md  |
| scripts/    | 可静态重复运行的动作，例如验证脚本、fixture generator、deterministic probe。       | scripts/check_env.py       |
| assets/     | 示例 JSON、静态资源、样例输入等。                                            | assets/example-config.json |

9. Background Review 触发与执行流程

Background Review 是回合后的复盘分支。它在主任务完成后，根据工具迭代数和会话轨迹决定是否 fork 一个后台 Agent 进行判断；fork 后也不一定写 Skill。



**creation\_nudge\_interval** 在配置示例中被描述为每 N 次 tool-calling iterations 提醒模型考虑保存 Skill；这不是“每调用一个 Skill 加一”，而是工具调用复杂度信号。

10. 写入安全边界：为什么不会任意乱改

| 机制                         | 作用                                                                   |
|----------------------------|----------------------------------------------------------------------|
| read-before-write          | Background Review 要 patch/edit/write 已存在目标前，必须先通过 skill_view 加载目标内容。 |
| external/bundled/hub guard | 后台自治分支拒绝修改 externally owned、bundled、hub-installed 等受保护 Skill。        |
| pinned guard               | Pinned Skill 对后台维护更严格，防止无用户在场时被自治修改。                                 |
| write approval gate        | 若配置开启，写操作先 stage 到 pending 队列，用户 review/diff/approve/reject 后再落盘。    |
| security scan              | agent-created Skill 默认不扫描；可通过 guard_agent_created 开启扫描，阻断危险内容。       |
| cache invalidation         | 写入成功后清理 skills system prompt cache 与 snapshot，使后续索引重建。               |

11. 例子：如何判断新建还是修改

11.1 应该修改已有 Skill 的例子

本轮任务：调试 Python 项目。  
已加载：python-debug。  
发现：项目有 uv.lock，应该优先 uv sync，而不是 pip install。  
判定：这是 python-debug 覆盖的类别。  
动作：patch python-debug 的 Procedure/Pitfalls，或写 references/uv-workflow.md。  
不应：新建 uv-lock-debug-fix 这种碎片 Skill。

11.2 应该写支持文件的例子

本轮任务：部署 Docker Compose 服务。  
发现：Compose 网络、localhost、service name 有一组细节规则。  
已有 umbrella：docker-troubleshooting。

判定：主流程只需一句触发规则，细节较多。  
动作：write\_file references/compose-networking.md，并在 SKILL.md 添加指针。

11.3 可以新建 Skill 的例子

本轮任务：反复完成 Kubernetes 蓝绿发布和回滚流程。  
现有 Skill：python-debug、docker-troubleshooting、github-pr-workflow。  
判断：没有已有 Skill 覆盖该 class。  
动作：create k8s-blue-green-deployment，category=devops 或 mlops 等 sensible category。  
要求：名字必须是 class-level，而不是 fix-prod-rollout-2026-07-09。

12. 研究口径总结

| 概念                | 准确表述                                    |
|-------------------|-----------------------------------------|
| 类别归属              | LLM-guided semantic placement，不是硬编码分类器。 |
| 新建 Skill          | 只有没有已有 class-level Skill 覆盖时才创建。        |
| 修改 Skill          | 优先当前已加载 Skill，其次已有 umbrella。            |
| 支持文件              | 承载细节、模板和可运行脚本，避免 SKILL.md 膨胀。           |
| Background Review | 触发后做判断，不等于一定写入。                         |
| 自进化               | 文件级程序性记忆更新，不是模型参数训练。                    |

13. 源码证据来源

以下为本文依据的源码文件与关键证据点。建议阅读顺序：background\_review.py → learn\_prompt.py → skill\_manager\_tool.py → cli-config.yaml.example。

| 文件                          | 关键证据点                                                                                                                                           | 源码链接                                                                                                                                                                                                    |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| agent/background_review.py  | Background Review fork、学习信号、优先级顺序、禁止沉淀项、class-level umbrella、support file 规则、protected skill 规则。                                                | <a href="https://raw.githubusercontent.com/NousResearch/hermes-agent/main/agent/background_review.py">https://raw.githubusercontent.com/NousResearch/hermes-agent/main/agent/background_review.py</a>   |
| agent/learn_prompt.py       | /learn prompt、Skill authoring standards、description 60 字符、body section order、Pick a sensible category。                                          | <a href="https://raw.githubusercontent.com/NousResearch/hermes-agent/main/agent/learn_prompt.py">https://raw.githubusercontent.com/NousResearch/hermes-agent/main/agent/learn_prompt.py</a>             |
| tools/skill_manager_tool.py | skill_manage schema；create/update 条件；patch/edit/write_file/delete 动作；category 校验；read-before-write；write gate；security scan；cache invalidation。 | <a href="https://raw.githubusercontent.com/NousResearch/hermes-agent/main/tools/skill_manager_tool.py">https://raw.githubusercontent.com/NousResearch/hermes-agent/main/tools/skill_manager_tool.py</a> |
| cli-config.yaml.example     | skills.creation_nudge_interval；external_dirs；工具迭代提醒；local skills precedence。                                                                    | <a href="https://raw.githubusercontent.com/NousResearch/hermes-agent/main/cli-config.yaml.example">https://raw.githubusercontent.com/NousResearch/hermes-agent/main/cli-config.yaml.example</a>         |

具体行号对应本文引用：background\_review.py L10-L35、L14-L18、L27-L30；learn\_prompt.py L3-L18；skill\_manager\_tool.py L32-L38、L65-L88；cli-config.yaml.example L66-L69。

附录：一句话背诵版

Hermes 的 Skill 自进化不是训练模型，而是由 Background Review Agent 根据源码提示词判断本轮经验是否可复用；判断时优先 patch 本轮已加载 Skill，其次 patch 已有 umbrella Skill，再把细节写入 references/templates/scripts，最后才创建新的 class-level Skill。类别归属由 LLM 选择 sensible category，源码只做 category 名称、Frontmatter、路径、安全和写入校验。